TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA

DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:

ENERO - AGOSTO 2026

CARRERA:

INGENIERÍA EN SISTEMAS COMPUTACIONALES

MATERIA:

PATRONES DE DISEÑO

UNIDAD A EVALUAR:

II

NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO:

ESMERALDA GAMBOA HERNANDEZ - 21210374

VELAZQUEZ MORALES LUCERO - 22210362

VIDALES HURTADO ALITZEL DALHI - 21212371

VALDES SIQUEIROS IVAN ANTONIO -

NOMBRE DE MAESTRO(A):

GUERRERO LUIS MARIBEL

Índice

Introducción **2**
Desarrollo del sistema **2**
Fase 1: Desarrollo de interfaces y Diagrama UML 2
Fase 2: Implementación de fábricas y productos 7
Conclusiones **10**

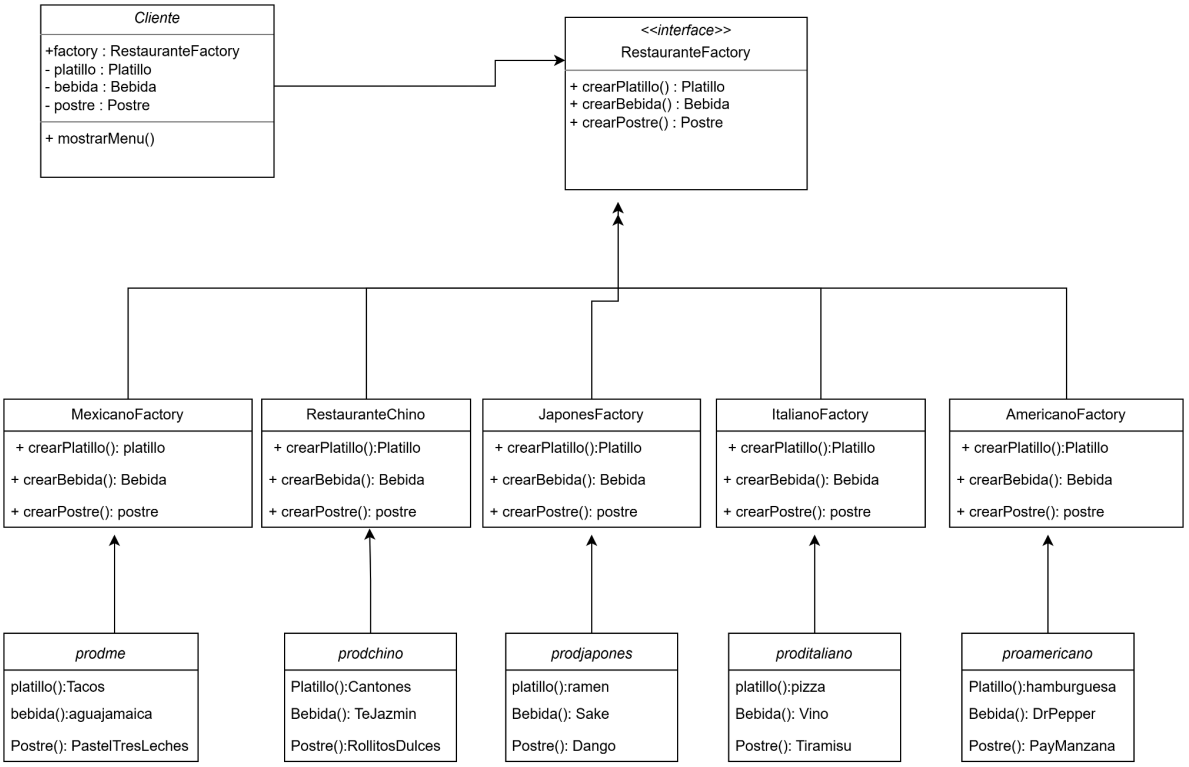
Introducción

La fábrica abstracta es un patrón de diseño que nos permite crear una familia de objetos relacionados entre sí dividiéndolos por categorías, en este caso estaremos implementando este patrón para crear un sistema de un restaurante que ofrece diferentes tipos de comidas segun su categoria (Mexicana, Italiana, Japonesa, China, etc.) . Al aplicar la fábrica abstracta en un sistema de restaurante estamos creando un sistema que nos permite gestionar de forma organizada y escalable la creación de componentes asociados a distintas categorías de comida mientras aseguramos que todas las piezas del sistema sean compatibles entre sí dentro de una misma familia.

Desarrollo del sistema

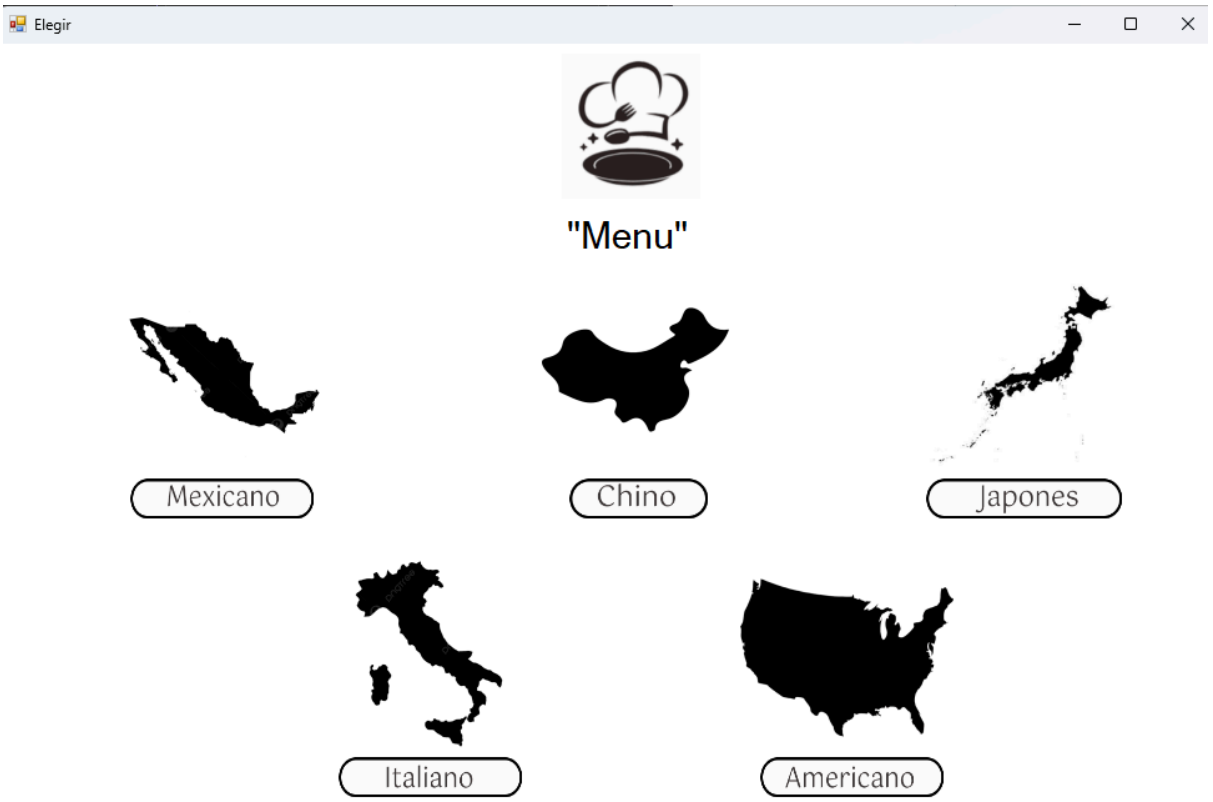
Fase 1: Desarrollo de interfaces y Diagrama UML

Como primera fase de la creación del sistema empezamos con el **Diagrama UML** para conocer los requisitos, clases, objetos y la relación que existirá entre todos los componentes que sean necesarios para nuestro código y poder empezar a diseñar visualmente en base a este diagrama.



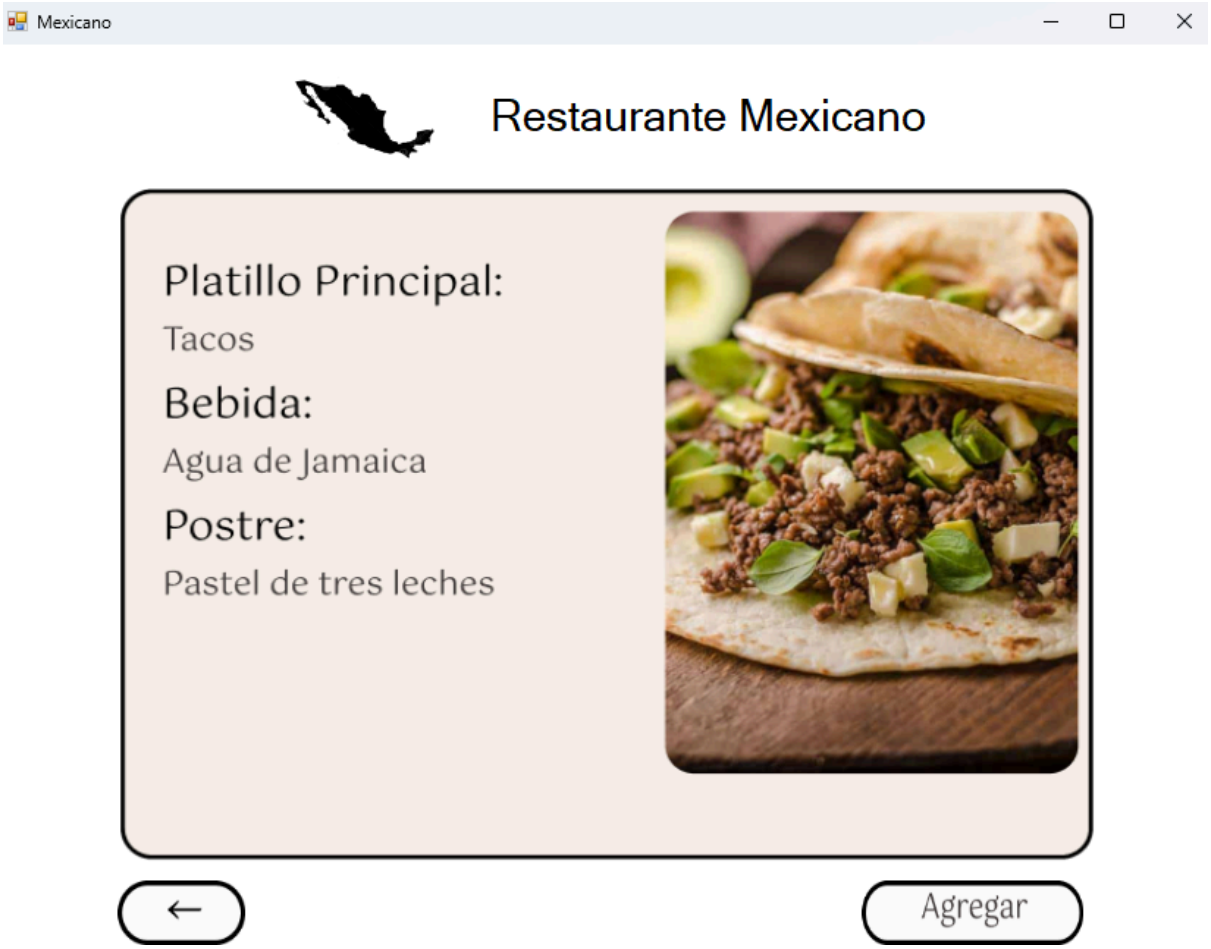
En nuestro diagrama UML podemos observar la fábrica central la cual es el restaurante en sí, los diferentes tipos de comida son las categorías o familia de objetos que encontramos dentro de nuestra fábrica central, cada tipo de comida diferente ofrece tres diferentes productos los cuales son los más populares de cada categoría correspondiente.

En base al diagrama UML presentado empezamos con el **Diseño de interfaces** que es nuestra base para cuando empecemos a realizar código podamos tener una mejor perspectiva de cómo va a interactuar el usuario con nuestro sistema y lo podamos contemplar en la parte lógica del código.



Primero tenemos la página de inicio en la cual ingresamos directamente al restaurante y le damos diferentes opciones de tipos de comida al usuario.
Dependiendo de la opción que haya escogido el usuario se mostrará una de las siguientes interfaces en las cuales entrará a la categoría elegida anteriormente, dentro de esta interfaz podrá escoger los productos que desee añadir a su pedido,

una vez finalizada la elección del usuario, podrá realizar el pedido y volver al menú principal.



Restaurante Chino

Platillo Principal:

Cantones

Bebida:

Te Jazmin

Postre:

RollosDulces

← Agregar

Restaurante Japones

Platillo Principal:

Ramen

Bebida:

Sake

Postre:

Dango

← Agregar

Restaurante Italiano

Platillo Principal:

Pizza

Bebida:

Vino

Postre:

Tiramisu

←

Agregar

Restaurante Americano

Platillo Principal:

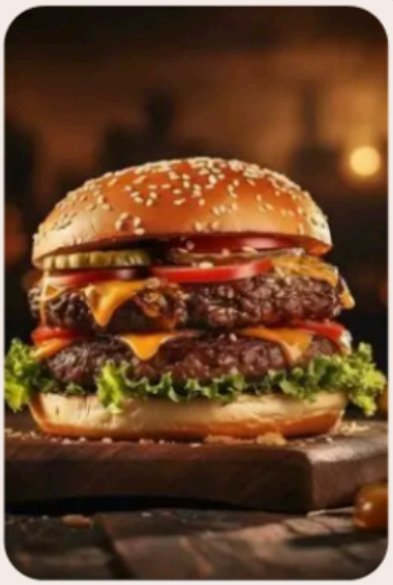
Hamburguesa

Bebida:

DrPepper

Postre:

PayManzana



←

Agregar

Fase 2: Implementación de fábricas y productos

Ahora que tenemos una base sólida de información podemos empezar a realizar el código de la parte lógica de nuestro programa, crearemos la fábrica restaurante.

```
namespace Fabrica_abstracta
{
    10 referencias
    public abstract class Restaurantes
    {
        10 referencias
        public abstract Comida PedirComida();
        10 referencias
        public abstract Bebida ServirBebida();
        10 referencias
        public abstract Postre OrdenarPostre();
    }
```

Esta clase representa la fábrica abstracta la cual define tres métodos abstractos:

1. PedirComida()

```
public abstract class Comida
{
    10 referencias
    public abstract void PedirC();
}
```

2. ServirBebida()

```
public abstract class Bebida
{
    10 referencias
    public abstract void ServirB();
}
```

3. OrdenarPostre()

```
public abstract class Postre
{
    10 referencias
    public abstract void OrdenarP();
}
```

Cada restaurante que herede de esta clase debe implementar estos métodos y decidir qué tipo de comida, bebida y postre ofrecer. Así hacemos que cada restaurante cree su propio menú.

Para esto primero heredamos de la clase Comida a la clase Tacos con el método Pedir();

```
//Comida Mexicana
1 reference
public class Tacos : Comida
{
    6 references
    public override void PedirC()
    {

        MessageBox.Show("Comida: Tacos");


    }
}
```

y lo mismo para bebida, se hereda de la clase Bebida a la clase pública Agua Jamaica y agregamos el método ServirB();

```
1 reference
public class AguaJamaica : Bebida
{
    6 references
    public override void ServirB()
    {
        MessageBox.Show("Bebida: Agua de Jamaica");
    }
}
```

Por último para la clase PastelTresLeches se hereda de Postre y se le agrega el metodo OrdenarP();

```
public class PastelTresLeches : Postre
{
    6 references
    public override void OrdenarP()
    {
        MessageBox.Show("Postre: Pastel de Tres Leches");
    }
}
```

Repetimos este paso con todos los tipos de comida hasta llegar al punto de crear las fábricas concretas. Lo que queremos es categorizar los tipos de restaurantes (Mexicano,Chino,Japonés,Italiano,Americano) y eso lo haremos con la herencia de la clase Restaurante.

```
//Fabricas concretas
1 reference
public class RestauranteMexicano : Restaurantes
{
    6 references
    public override Comida PedirComida()
    {
        return new Tacos();
    }
    6 references
    public override Bebida ServirBebida()
    {
        return new AguaJamaica();
    }
    6 references
    public override Postre OrdenarPostre()
    {
        return new PastelTresLeches();
    }
}
```

Agregamos los métodos con un public override que funciona para se usa para redefinir un método que viene de una clase base o interfaz, permitiendo que cada fábrica concreta cree sus propios objetos.

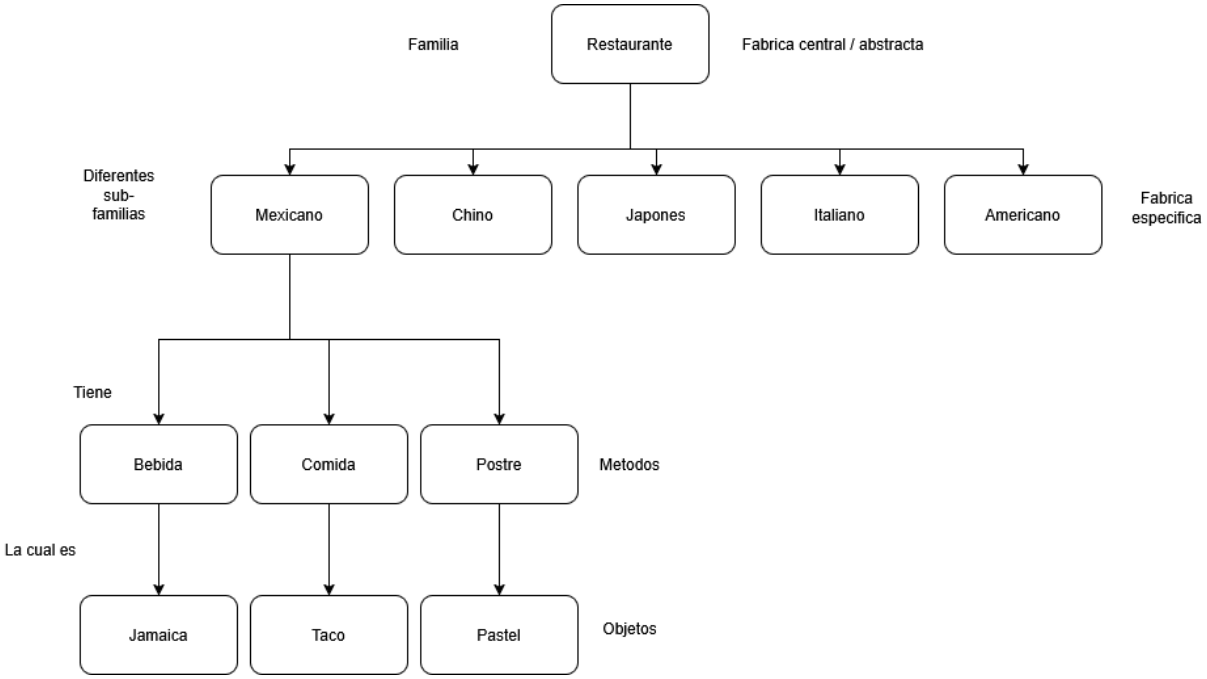
Implementación específica para forms es la siguiente

```
private void button1_Click(object sender, EventArgs e)
{
    Restaurantes chino = new RestauranteJapones();

    Comida comida = chino.PedirComida();
    Bebida bebida = chino.ServirBebida();
    Postre postre = chino.OrdenarPostre();

    comida.PedirC();
    bebida.ServirB();
    postre.OrdenarP();
}
```

Diagrama de implementación



Conclusiones

ESMERALDA GAMBOA HERNANDEZ

En conclusión el método abstract factory sirve para generar objetos que sean de la misma familia, osea que sean parecidos pero no iguales. Se crea la Fábrica Abstracta (Restaurantes) y después se agregan Clases abstractas (Comida,Bebida,Postre) que contienen un método para generar los objetos. Las fábricas concretas se heredan de la fábrica abstracta y generan las diferentes comidas, bebidas y postres dependiendo del tipo de restaurante. Fue una práctica interesante, pude entender mejor el patrón de manera más detallada y fue divertido trabajar en equipo.

VELAZQUEZ MORALES LUCERO

En este proyecto utilizamos el patrón abstract factory para organizar la creación de los diferentes elementos del menú de cada restaurante con este patrón pudimos separar cada tipo de restaurante, cómo mexicana, china, etc y a sí asegurar que los platillos, bebidas y postre pertenezcan a la misma categoría sin mezclarse entre sí.
en conclusión el patrón abstract factory nos permitió tener un sistema más ordenado y fácil de entender, ya que cada restaurantes funciona como una fábrica que crea su propio menú.

VIDALES HURTADO ALITZEL DALHI

Abstract Factory disminuye significativamente mucho tiempo de desarrollo ya que con solo crear un objeto puedes crear n cantidad de estos mismos cambiando muy pocas líneas de código, también es muy fácil y rápido relacionarse con la manera de programación que requiere este patrón de diseño. La lógica de programación que se maneja dentro de este patrón es bastante sencilla al manejar objetos similares que cambian en pocos aspectos, lo cual nos sirvió bastante a la hora de desarrollar el código del restaurante y las diferentes tipos de comidas que ofrecia.

VALDES SIQUEIROS IVAN ANTONIO

El patrón abstract factory puede llegar a ser muy útil cuando se requiere crear múltiples objetos que comparten categoría pero que son diferentes en métodos y atributos por que pueden llegar a hacer cosas parecidas pero no lo mismo así ahorrando tiempo de desarrollo, pero también agrega complejidad comparado con sin patrón por que es un esquema rígido a seguir que puede llegar a complicar el entendimiento general del programa y el desarrollo.